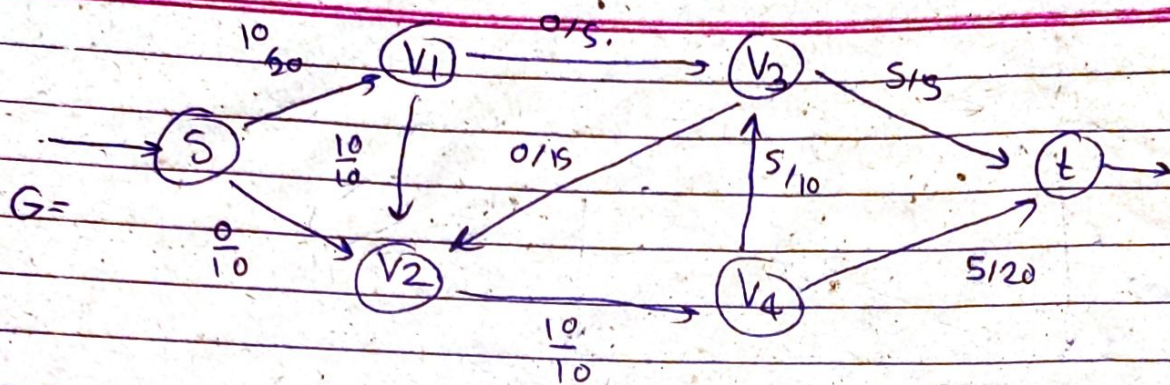
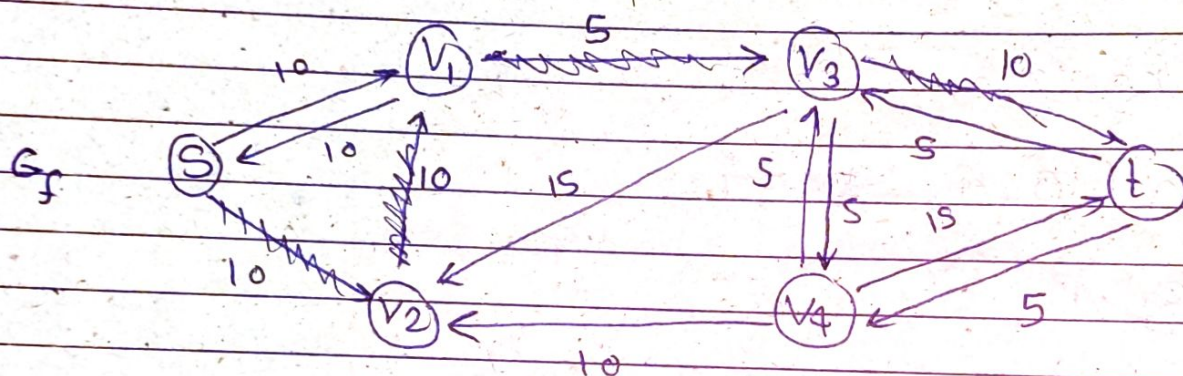




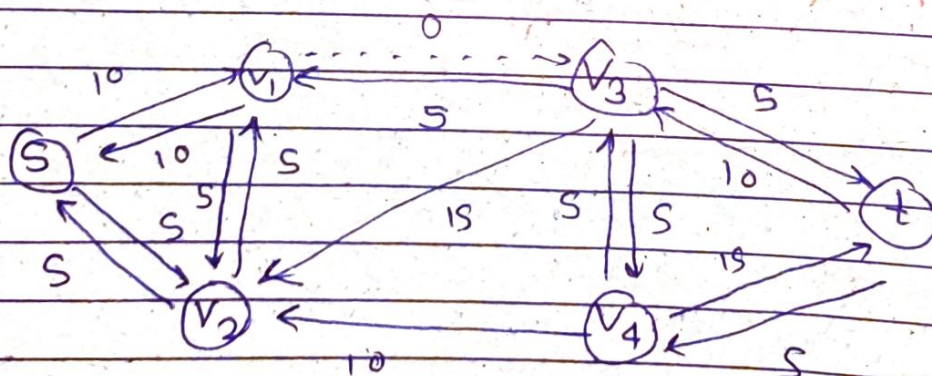
Making Residual Graph



Taking path $S \rightarrow V_2 \rightarrow V_1 \rightarrow V_3 \rightarrow t$

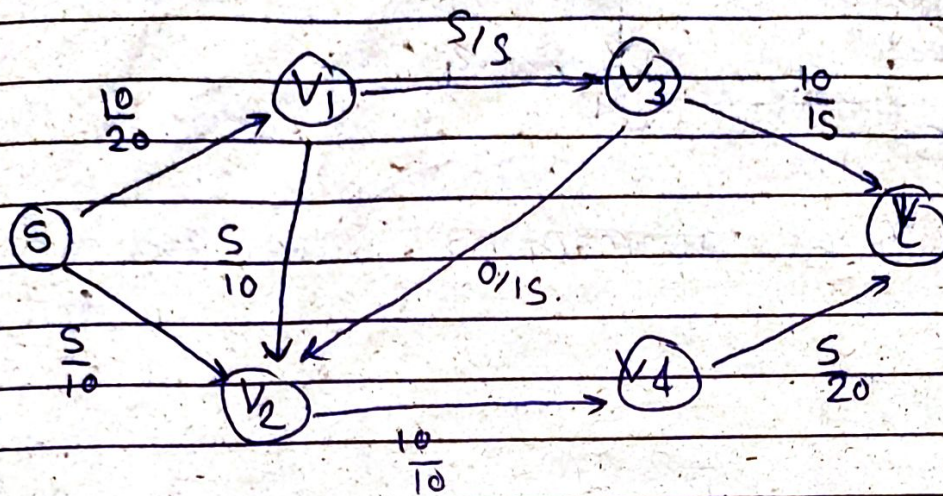
$$\min(10, 10, 5, 10) = 5$$

Updated $G_f =$

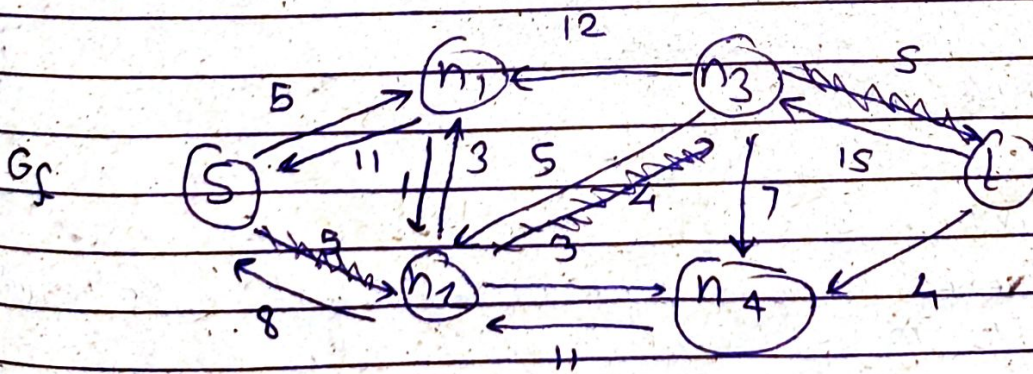
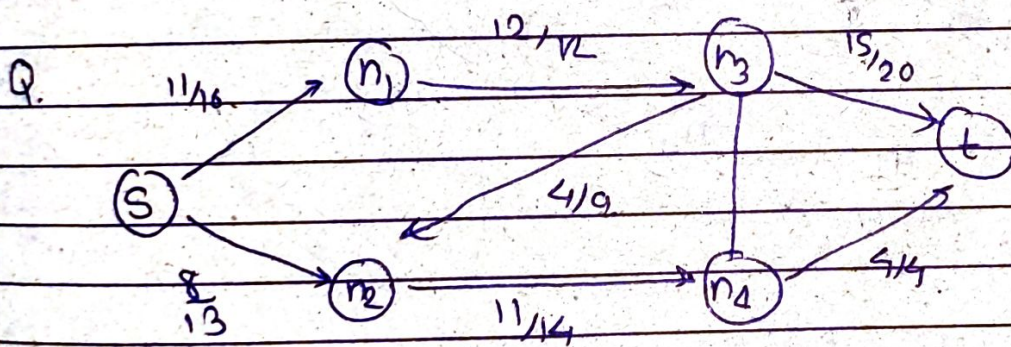


No more augmenting Paths

Reverting back to graph.

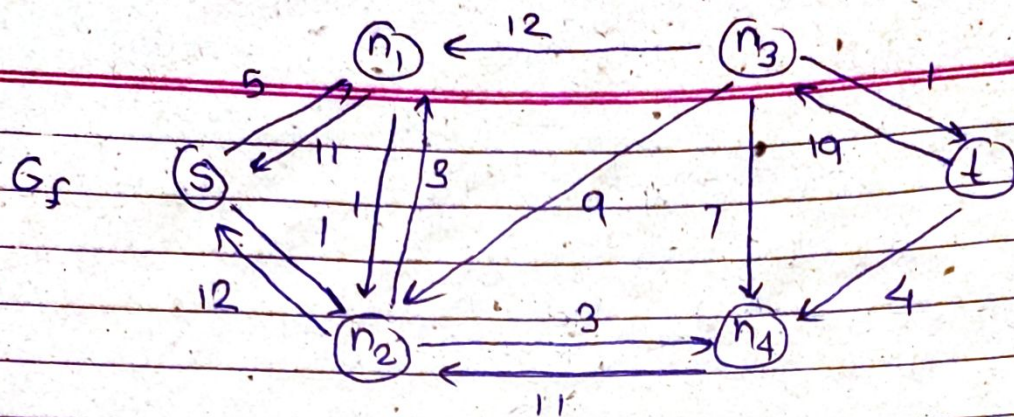


Total Maximum flow rate = 15

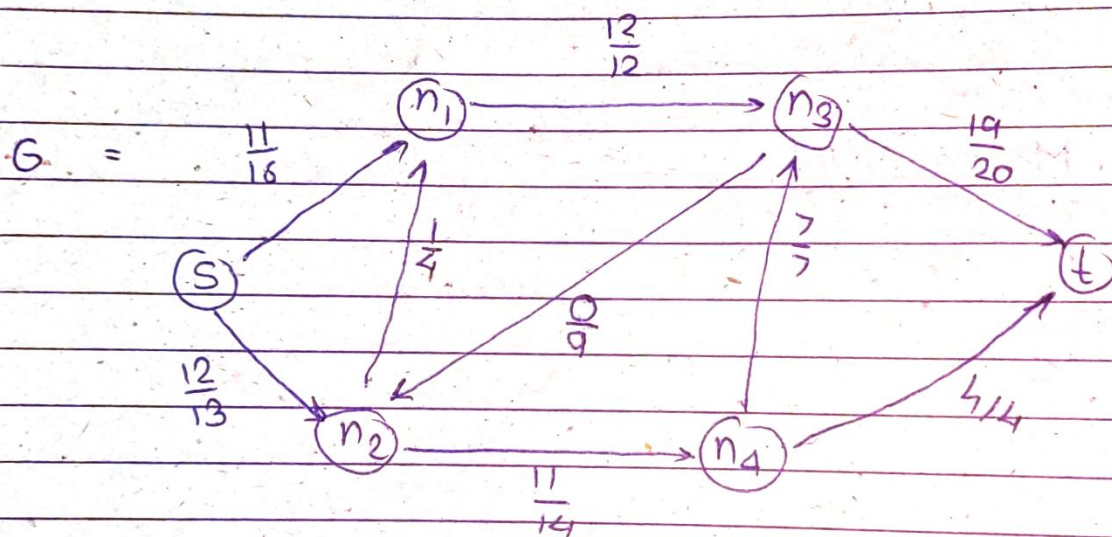


Taking Path S, n_1, n_3, t

$$\min\{5, 4, 5\} = 4$$



No more augmenting paths available.



Answer = 23

Ford Fulkerson (G, s, t)

for each edge $(u, v) \in G.E$
 $(u, v).f = 0$

while there exists a path p from s to t in G_f (residual)

$C_f(p) = \min \{ C_f(u, v) : (u, v) \text{ is in } p \}$

for each edge (u, v) in p

if $(u, v) \in E$

$(u, v).f = (u, v).f + C_f(p)$

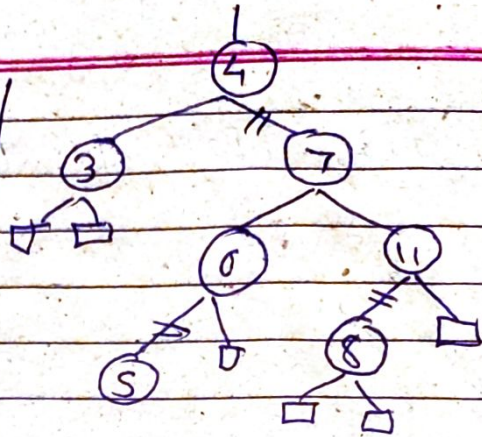
else $(v, u).f = (v, u).f - C_f(p)$

Accounting Method:

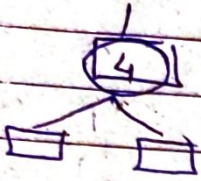
Counter Value	A[3]	A[2]	A[1]	A[0]	Amortized Cost	Actual Cost	Credit
0	0	0	0	0	0	0	0
1	0	0	0	1	2	1	1
2	0	0	1	0	2	2	1
3	0	0	1	1	2	1	2
4	0	1	0	0	2	3	1
5	0	1	0	1	2	1	2
6	0	1	1	0	2	2	2
7	0	1	1	1	2	1	3
8	1	0	0	0	2	4	1

Counter	A[3]	A[2]	A[1]	A[0]	Actual Cost	$\phi(D_i)$	$\phi(D_{i-1})$	Potential diff	Amortized Cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	1	1	0	1	2
2	0	0	1	0	2	1	1	0	2
3	0	0	1	1	1	2	1	1	2
4	0	1	0	0	3	1	2	-1	2
5	0	1	0	1	1	2	1	1	2
6	0	1	1	0	2	2	2	0	2
7	0	1	1	1	1	3	2	1	2
8	1	0	0	0	4	1	3	-2	2

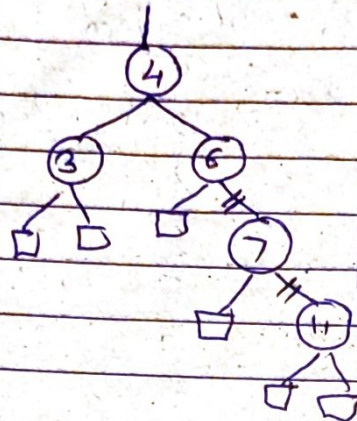
Insert 8.



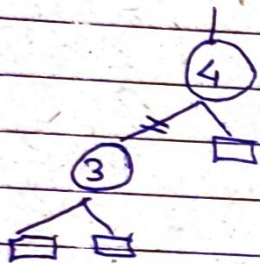
Insert 4.



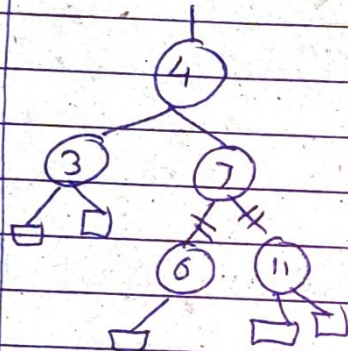
Insert 11.



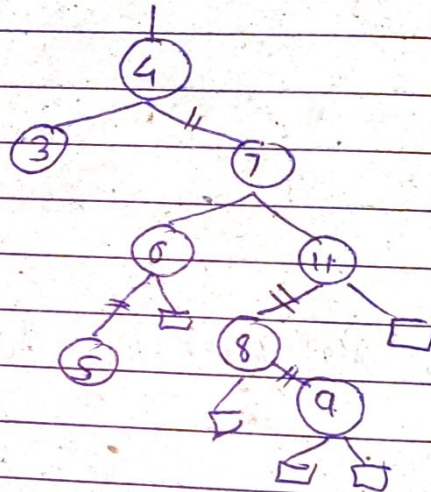
Insert 3



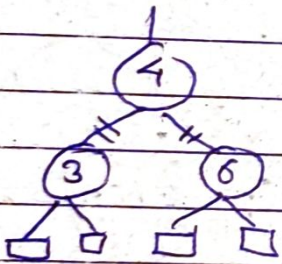
Single left rotation of 6.



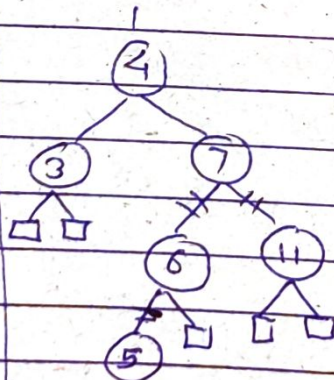
Insert 9.



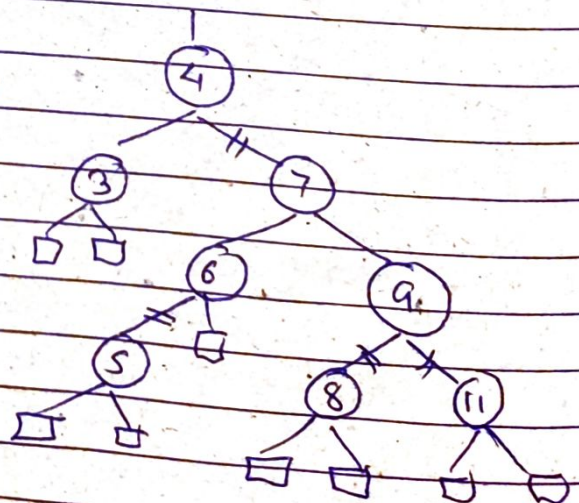
Insert 6.



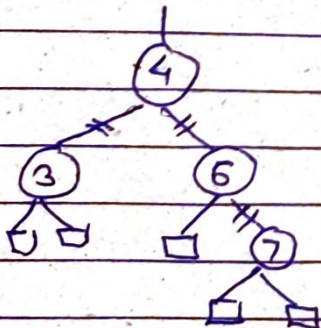
Insert 5.



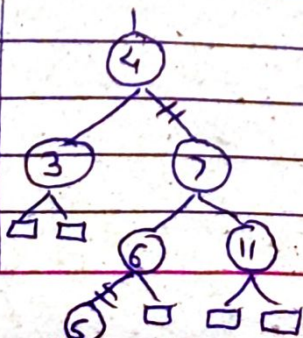
Double rotation.
Single left of 8
Single Rot of 11.



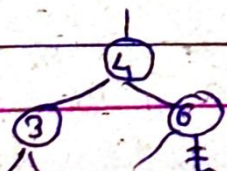
Insert 7.



Promotion.

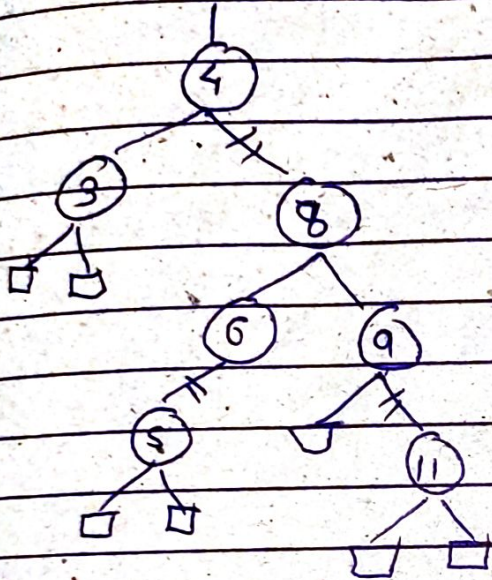


Promotion.



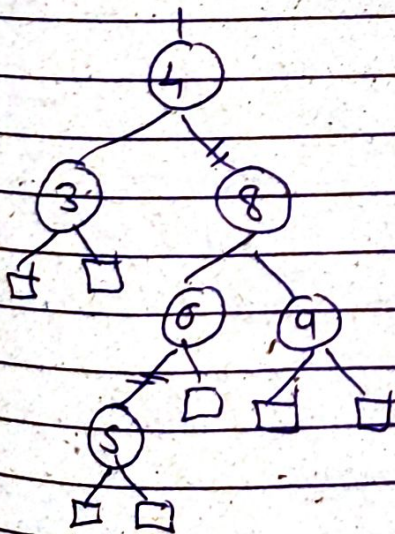
Remove 7

Swap 7 with 8
and remove.



Remove 11.

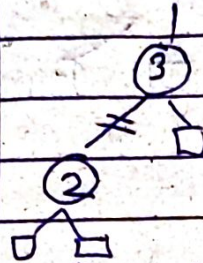
[No Violation]



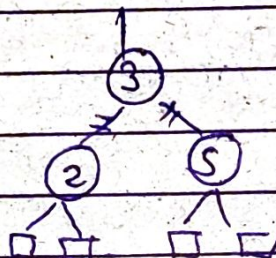
Insert 3



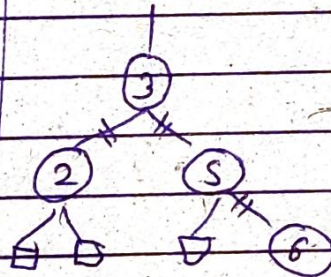
Insert 2



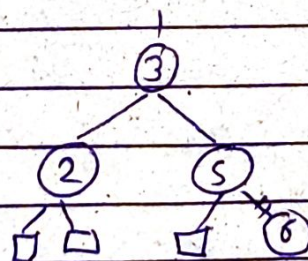
Insert 5



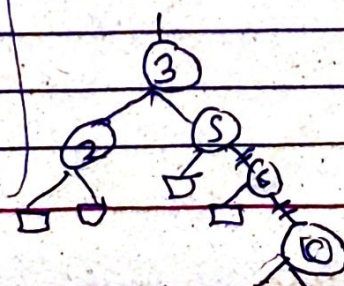
Insert 6



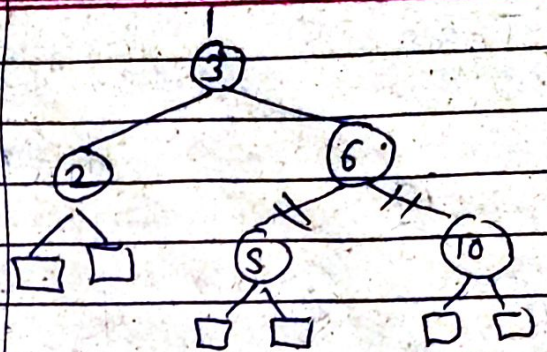
Promotion



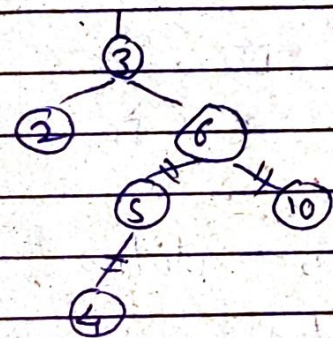
Insert 10



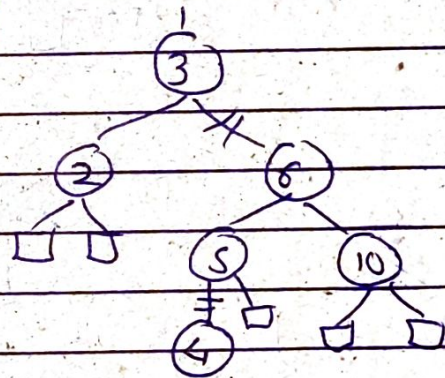
Single Left Rotation
of 5.



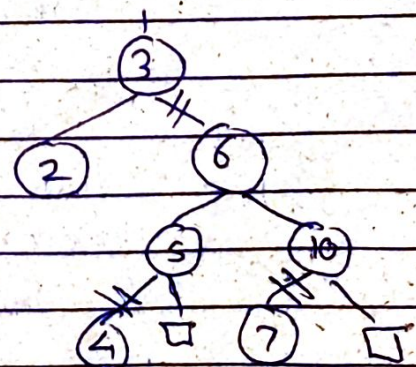
Insert 4



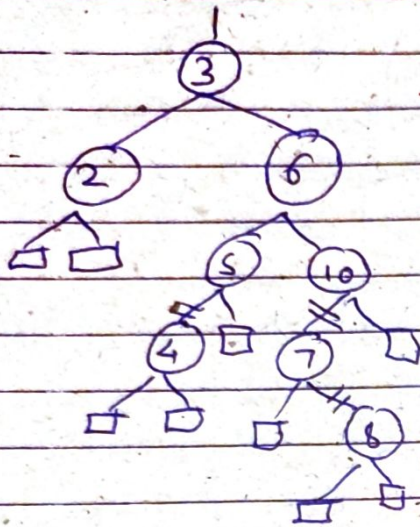
Promotion



Insert 7

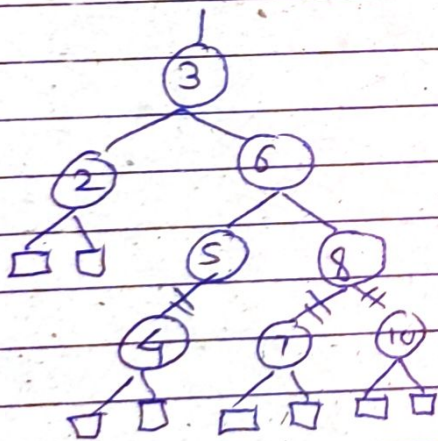


Insert 8.



Double rotation

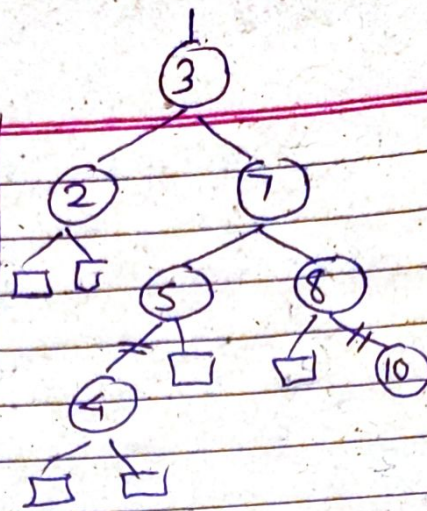
- Singl left of 7
- Singl Right of 10.



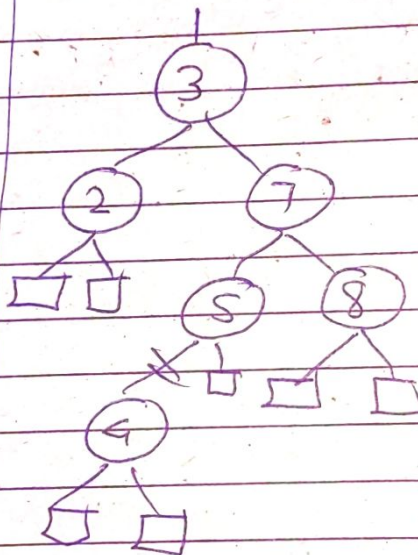
Remove 6.

Swap 6 with 7
(Successor).

and remove 6.



Remove 10
(No Violation)

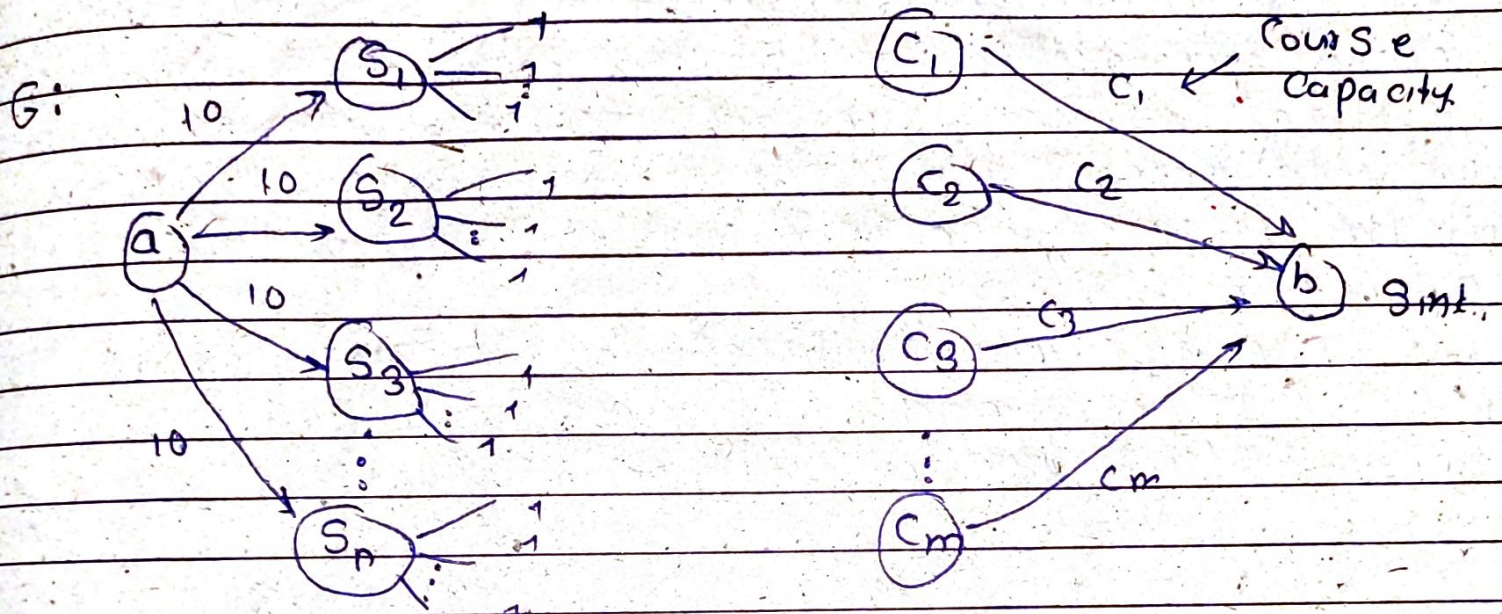


Q Consider the following problem of allocating students to classes. There are n students $S_1, S_2, \dots, S_n$ and m classes $C_1, C_2, \dots, C_m$. Each class C_j has its maximum enrollment capacity C_j . Each student S_i has a 1st L_i of 12 courses.

The university should enroll the student into 10 out of 12 required classes. The university should enroll

the student into The goal is to find an enrollment of the students to classes that satisfies the following conditions:

- The number of Students in each class C_i does not exceed c_i .
- Each student S_i is enrolled into exactly 10 class from list L .



For each student S_i there is a list of capacity 10.
 Edge (S_i, C_k) where C_k is one of the 10 courses student S_i likes.
 Find the max flow of G .

a Under what condition on the max-flow on the network from part (a) is it possible to give an enrollment as required by the problem?
 Give a brief explanation.

Possible : if value of max flow equals $10n$ (Each /Source S_i) edge must be saturated.

One unit of flow indicates that one student has been assigned to one course.

So, a flow with value $10n$ is required for n students to be assigned to courses each.

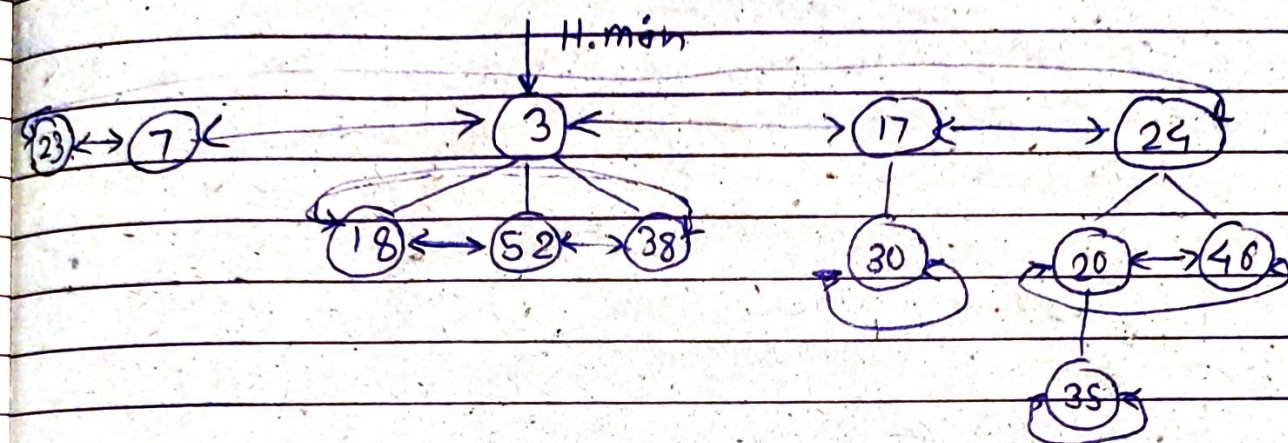
- * **Fibonacci Heap:** It is a data structure used for implementing priority queues. It is a type of heap data structure, but with several improvements over the traditional binary heap and binomial heap.
- The key advantage of Fibonacci heap is its fast amortized running time for operations such as insert, merge and extract-min, making it one of the most efficient data structures for these operations.
- The running time of these operations in a Fibonacci heap is $O(1)$ for insert, $O(\log n)$ for extract min and $O(1)$ amortized for merge.

	Fibonacci heap (amortized)
Make-Heap	$O(1)$
Insert	$O(1)$
Minimum	$O(1)$
Extract-Min	$O(\lg n)$
Union	$O(1)$
Decrease key	$O(1)$
Delete	$O(\lg n)$

- A Fibonacci heap is a collection of rooted trees that are min-heap ordered. That is, each tree obeys the min-heap property. (the key of node is greater than or equal to parent key)
- Each node x has pointer to $x.p$ parent and $x.c$ child.
- The children of x are linked together in a circular doubly linked list, which we call

the child list of x

- Each child y in a child list has pointer $y.left$ and $y.right$ to its siblings.
- If y is only child, then $y.left = y.right = y$.



We access a given fibonacci heap H by a pointer $H.min$ to the root of a tree containing the minimum key called the minimum node.

Parent	Child	key	degree	Mark
--------	-------	-----	--------	------

* Potential Function

- For a given Fibonacci heap H , we indicate by $t(H)$ the no of trees in the root list of H and by $m(H)$ the number of marked nodes in H .
- potential function $\phi(H)$ is given by

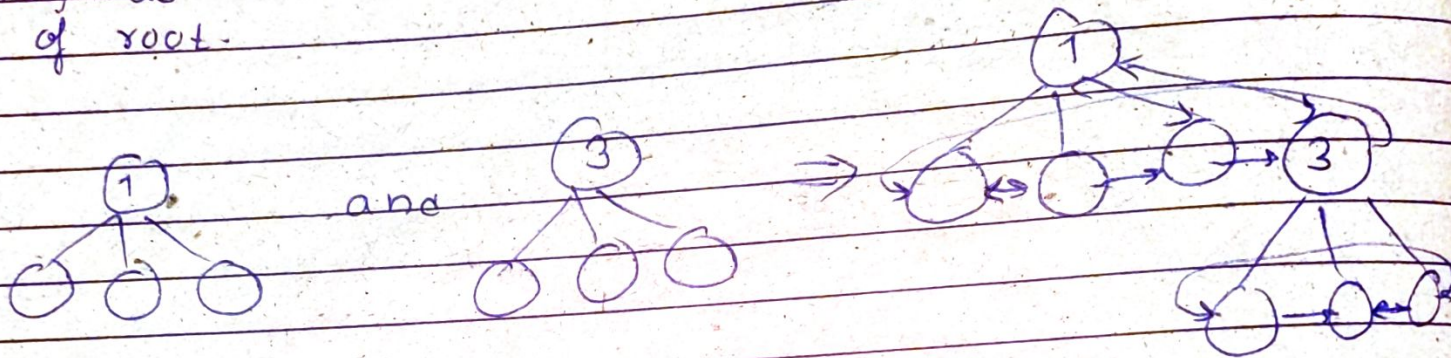
$$\phi(H) = t(H) + 2m(H)$$

③ P_i = Potential of data Structure after i^{th} operation.
 $= t_i(H) + 2m_i(H)$.

$$P.D = P_i - P_{i-1} = t_i(H) + 2m_i(H) - t_{i-1}(H) - 2m_{i-1}(H)$$

* Linking:

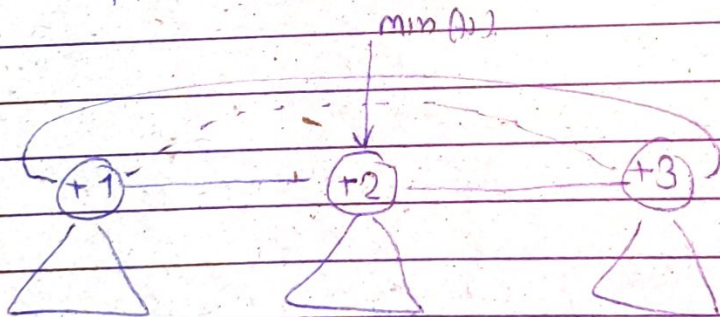
- Linking is a part of the consolidate() process which is triggered during extract min.
- Given two order trees, the one with smaller key as root and the other tree as child of root.



- Degree of parent is increased by 1.

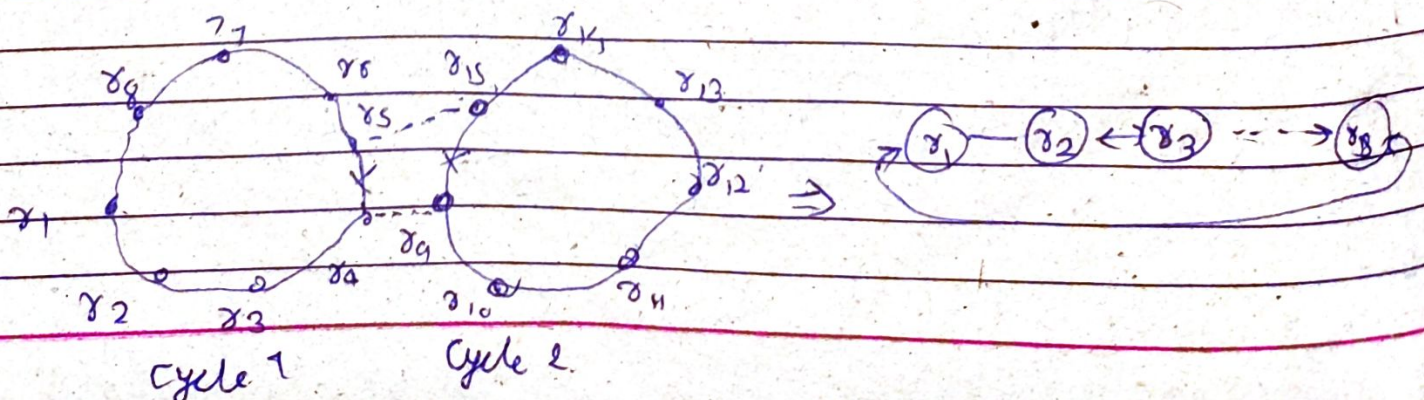
* Unlinking:

- It happens during decrease-key and cut.



* Merging of Cycles

- You can just concatenate the root lists.



* **Make_heap():**

- It creates a new empty fibonacci heap.
- Set min pointer to nullptr.
- Set node count $n=0$.

FibonacciHeap * MAKE_HEAP() {

FibonacciHeap * H = new FibonacciHeap();

H → min = nullptr;

H → n = 0;

return H;

}

Actual Cost = Amortized Cost = $O(1)$.

* **Minimum()**

It returns the minimum heap.

returns the pointer to the node with the minimum key in the heap H.

Node * Minimum (FibonacciHeap * H) {

return H → min;

}

• It does not affect no of nodes and no of max nodes.

* **Meld(H_1, H_2)** (Meld or Union)

• It merges two Fibonacci heaps, H_1 and H_2 into a new heap H.

Steps : 1] Merge the root cycle of H_1 and H_2 $\rightarrow O(1)$
 2] Adjust $\text{Min}(H)$ to be minimum ($\text{min}(H_1, \text{min}(H_2))$) $\rightarrow O(1)$

$\therefore \text{Actual Cost} = O(1) + O(1) = O(1)$

Fibonacci Heap * MELD (Fibonacci Heap * H_1 , H_2)

if ($H_1 \rightarrow \text{min} == \text{nullptr}$) return H_2

if ($H_2 \rightarrow \text{min} == \text{nullptr}$) return H_1

Node $\rightarrow H_1 \rightarrow \text{right} = H_1 \rightarrow \text{min} \rightarrow \text{right}$

Node $\rightarrow H_2 \rightarrow \text{left} = H_2 \rightarrow \text{min} \rightarrow \text{left}$

$H_1 \rightarrow \text{min} \rightarrow \text{right} = H_2 \rightarrow \text{min}$

$H_2 \rightarrow \text{min} \rightarrow \text{left} = H_1 \rightarrow \text{min}$

$H_1 \rightarrow \text{right} \rightarrow \text{left} = H_2 \rightarrow \text{left}$

$H_2 \rightarrow \text{left} \rightarrow \text{right} = H_1 \rightarrow \text{right}$

$H_1 \rightarrow \text{min} = \max(H_1 \rightarrow \text{min} \rightarrow \text{key}, H_2 \rightarrow \text{min} \rightarrow \text{key})$

$H_1 \rightarrow n += H_2 \rightarrow n$

return H_1

3.

Cost = $O(1) + O(1) = O(1)$

* Insert (H, x)

Steps -

1. Create a new heap H_2 with a single node x .

2. using $\text{make_heap}(x)$

3. $\text{meld}(H, H_2)$ Merge the root list of H_1

which contains just x into root list of H .

3. Update the min pointer to the resulting heap node which has smaller key between $H.min$ and x

insert(H, x):

$H1 = \text{make_heap}(x)$

$H = \text{meld}(H, H1)$

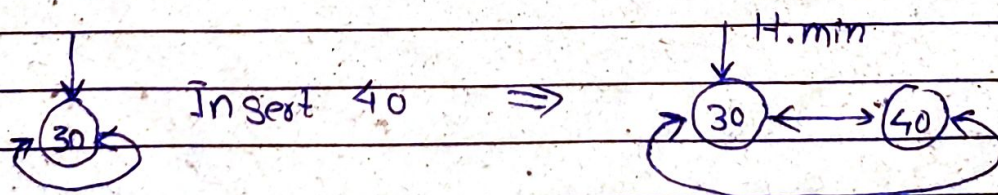
$$\begin{aligned} \text{Potential difference} &= \phi(H_{\text{after}}) - \phi(H_{\text{before}}) \\ &= \sum_{i=1}^n [H_i] + 1 + 2 \cdot m_{i-1}(H) - t_{i-2} + 2 m_{i-1}(H) \end{aligned}$$

and

$$\begin{aligned} &= t_{\text{before}} + 1 + 2m_{\text{before}} - t_{\text{before}} + 2m_{\text{before}} \\ &= 1 \end{aligned}$$

$$\begin{aligned} \Rightarrow \text{Amortized Cost} &= O(1) + \Delta\phi \\ &= O(1) + 1 \\ &= \underline{\underline{O(1)}} \end{aligned}$$

* ~~General~~



• During insert all elements are added to root cycle.

* Delete Min ()

- The delete min operation removes the node with the minimum key from the heap and performs consolidation to maintain heap efficiency.
- This operation is non-trivial and costs $O(\log n)$.

Steps:

1. Remove the node with min key from the root cycle.
2. Merge the cycle of children of the node with root cycle.
3. While the two roots x_1, x_2 have same degree do link x_1, x_2 .
4. Adjust minimum key.

Decrease

Let $z = H.min$

for each child x of z :

 Add x to root cycle

 Set $x.p = null$

remove z from root cycle.

consolidate: link roots with same degree.

Set $H.min$ to min key in root list.

* Decrease_key(H, x, Δ)

H is the fibonacci heap, x is a pointer to the node you want to decrease, by Δ .

- Steps :
1. Unlink tree rooted at x
 2. Decrease key by Δ
 3. Add x to original root cycle.
 4. Do cascading cut on $y = x.p$

- When we decrease the key, we need to check whether the heap property is violated.
- If violates then the node is cut from its parent and added to root cycle.
- If a node x loses its first child its mark bit is set to true.
- If it loses second child (i.e. if its mark bit is already true) then cut the node from its parent and attach to root cycle.
- Make node's mark bit false.

• $O(\log n)$

* Delete (H, x) .

- To delete a node x we first decrease the key of x to a value smaller than any other key (i.e. smaller than $h.min$).
- So, x becomes new $h.min$.
- Then we call $delete_min(H)$.

NP - Completeness

- The class P consists of those problems that are solvable in polynomial time $O(n^k)$ for some constant k .

- The class NP consists of those problems that are "verifiable" in polynomial time. If we were given a "Certificate" of a solution, then we can verify that the certificate is correct in time polynomial in the size of input to the problem.

- Any problem in P is also in NP, since if a problem is in P then we can solve it in polynomial time without even being supplied a certificate.

$$P \subseteq NP.$$

- A problem is in the class NPC (NP Complete) if it is in NP and is as hard as any problem in NP.

* Decision Problem Vs Optimization problem

Many problems of interest are optimization problems, in which each feasible solution has an associated value and we wish to find a feasible solution with the best value.

For example:

In a problem of Shortest-Path, we are given an undirected graph G and vertices u and v and we wish to find a path from u to v that uses fewest edges.

- NP-Completeness applies directly not to optimization problems, however, but to decision problems, in which the answer is simply "yes" or "no".

- We can usually cast a given optimization problem as a related decision problem by imposing a bound on the value to be optimized.

ex. Given a directed graph G , vertices u , and v and an integer k , does a path exist from u to v consisting of at most k edges.

* Reductions:

- Consider a decision problem A , which we would like to solve in polynomial time.
- We call the input to a particular problem an instance of that problem.
- Now suppose that we already know how to solve a different decision problem B in polynomial time.
- Finally, suppose that we have a procedure that transforms any instance α of A into some instance β of B .

with the following characteristics:

- The transformation takes polynomial time.

- The answers are the same, i.e. the answer for α is "yes" if and only if the answer for β is also "yes".

- We call such a procedure a polynomial time reduction algorithm.

* NP Completeness

NP Completeness is a concept in computational complexity theory that helps classify problems based on how difficult they are to solve and verify.

Types of Problem

- **Trackable**: Problems that can be solved in a reasonable (polynomial) time.
- **Intractable**: Some problems are intractable, as they grow large, we are unable to solve them in reasonable time.
- **Optimization problem**: An optimization problem is one which asks, what is the optimal solution to problem x .
ex. 0-1 Knapsack etc.
- **Decision Problem**: A decision problem is one with yes/no answer.
ex. Does a graph G have a MST of weight $\leq W$?

* The Class P

P: The class of problems that have polynomial time deterministic algorithms.

- That is they are solvable in $O(p(n))$, where $p(n)$ is a polynomial on n .
- A deterministic algorithm is (essentially) one that ~~always~~ always computes the correct answer.

ex. Fractional knapsack, MST, Sorting, Others.

* The Class NP.

The class of decision problems that are solvable in polynomial time on a non deterministic machine (or with non deterministic algorithm).

- Thus NP can also be thought of as the class of problems "whose solution can be verified in polynomial time."

ex. Travelling Salesman, SAT, Graph colouring, 3-CNF

$$P \subseteq NP$$

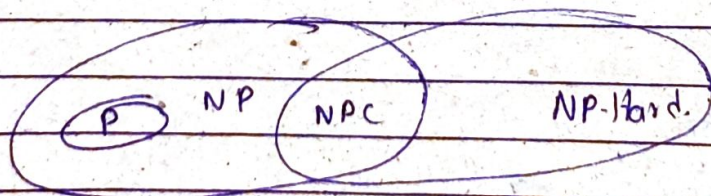
* NP-Hard.

A problem is NP-hard if all problems in NP are polynomial time reducible to it.

ex. Hamiltonian Cycle.

* NP-Complete

A problem is NP-Complete if the problem is both NP-Hard and NP.



* NP

NP is the class of decision problem for which a solution can be verified in polynomial time by a deterministic Turing machine.

ex. Sudoku: It is hard to fill grid, but easy to check a filled one.

Hamiltonian Cycle - Does a cycle visit every vertex once?

* NPC:

A problem is NP-Complete if it is

1. In NP (its solution is verified in polynomial time)

2. It is as hard as any problem in NP.
meaning, every NP problem can be reduced to it in polynomial time.

* NP-Hard:

A problem is NP-hard if

• It is at least as hard as any problem in NP

• Every problem in NP can be reduced to it in polynomial time.

• It may not be in NP (ie It may not be verifiable in polynomial time)

* Halting Problem:

It is one of the most famous problems, introduced by Alan Turing in 1936.

It asks whether, given a problem and its inputs, it is possible to determine whether the program will eventually halt (stop) or run forever.

The halting problem is not decidable, meaning no algorithm that can solve it for all possible programs and inputs.

* Hamiltonian Cycle Problem

• The Hamiltonian Cycle problem is a well known NP-Complete problem in graph theory.

• Given a graph G , does there exist a cycle that visits every vertex exactly once and returns to the starting vertex.

* Abstract Problem

An abstract problem is a relation $I \times S$

read CW

where:

I : { Set of Problems }

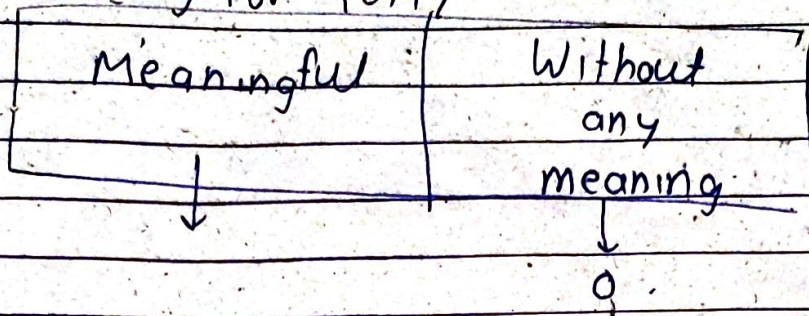
S : { Set of Solutions using some particular tool }

* Concrete Decision Problem

It is a decision problem with

$I = \{0, 1\}^*$

String for $\{0,1\}^*$



If a bit string does not correspond to a meaningful instance of a problem then the string is mapped to 0.